Reflective Graph: Software Architecture

Shahzada Saeed · MSc Computer Science

1. Purpose and design

Reflective Graph is a platform for constructing and investigating programs whose rules form part of their own data. A program consists of a graph containing ordinary relationships, descriptions of rewrite rules, and the information needed to apply those rules. Computation changes this structure. A rule may also change another rule, subject to conditions that keep each committed state valid.

This arrangement makes the boundary between a program and its data available for inspection. It supports experiments with self-modification, stable behaviour and the dependencies between computational events. The software combines an executable mathematical model with tools for writing programs, examining their execution and retaining reproducible results.

The central design decision is to keep the semantics in one Rust implementation. The command-line application, browser workbench and hosted service use the same carrier representation and admission rules. Interfaces differ in their resource allowances and available analyses. They do not supply alternative definitions of a valid rewrite.

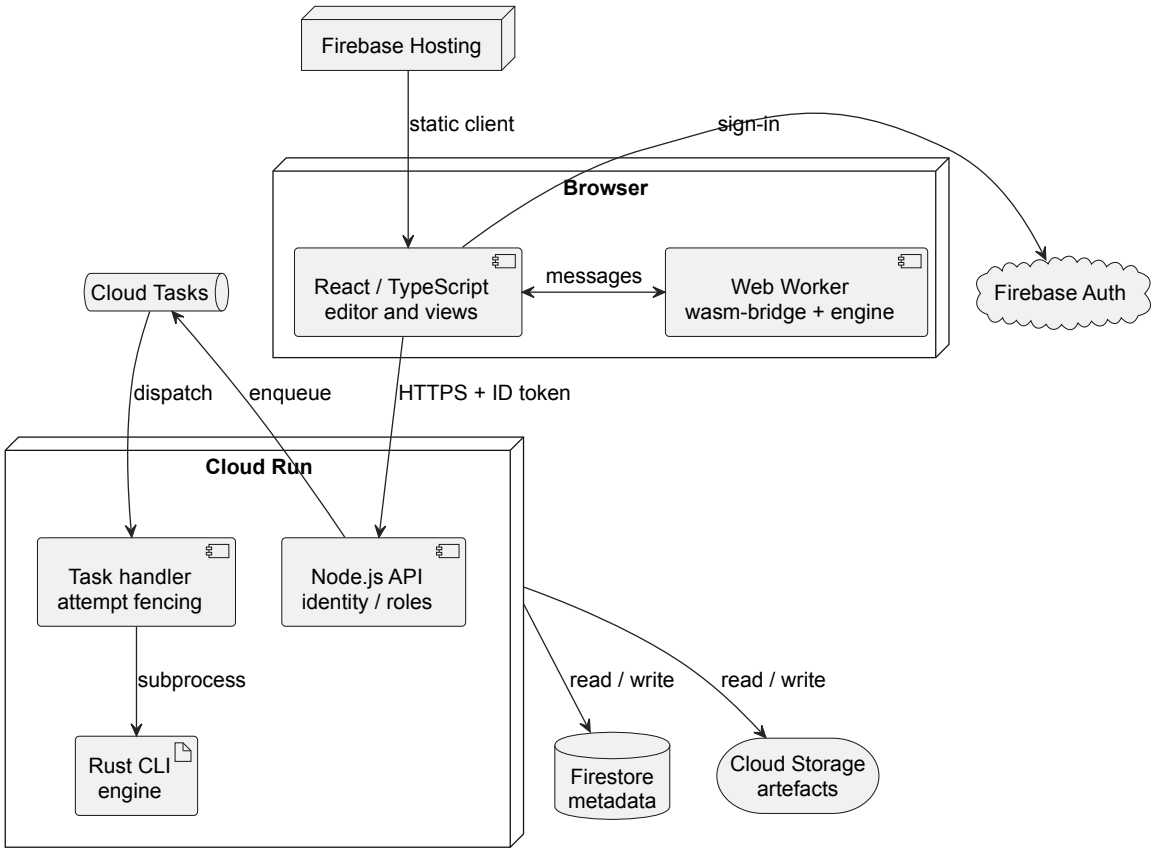


Figure 1. The software components and their execution boundaries. Browser execution and hosted execution use the same Rust semantics through different adapters.

2. Components and responsibilities

The Rust workspace contains five crates. The dependency direction keeps graph validity independent of the user interface and separates text processing from execution.

Component	Responsibility
`graph-core`	Atom identities, typed records, immutable fields, incidence indexes, graph validation and rule decoding.
`engine`	Matching, admission, atomic rewriting, scheduling, observation, recording and research analyses.
`parser`	Source syntax, diagnostics, reference resolution, graph construction and printing.
`cli`	Native program execution, trace verification, analysis commands and application adapters.
`wasm-bridge`	The browser-facing interface to the same graph, parser and engine code.

The engine and parser depend on `graph-core`; neither requires the other to define its main responsibilities. The CLI and WebAssembly bridge compose them. Research capabilities such as workflow analysis, numerical certificates, learned action models and provenance are modules within this workspace. They use ordinary graph values and the existing transaction boundary.

Outside Rust, the frontend uses React and TypeScript. A Node service supplies the project API, storage integration and job management. These layers handle interaction and persistence; they submit programs to Rust for validation and execution. Small independent evaluators used in testing remain separate from the authoritative engine so that comparisons can reveal implementation errors.

3. Graph representation and execution

3.1 One carrier, several interpretations

Every addressable element has an atom identity. Constructors represent labelled objects; record atoms represent relationships and rule metadata. A relationship has its own identity and refers to its endpoints through ordered fields. Rule heads, natural-number cells and pattern variables use the same carrier. Consequently, a rule can refer to a relationship or a rule description without introducing an edge whose endpoint is another edge.

Atom identities are 64-bit values and are never reused within a graph's allocation history. Fields and attributes are immutable. A changed relationship is represented by deleting its old record and producing a fresh record. The store uses a persistent ordered map and a reverse-incidence index. Structural sharing permits related states to retain unchanged data; reverse incidence supports checks for references to atoms being deleted.

The practical graph view draws object-link records as relationships between objects. The internal view exposes those records and their fields as atoms and arrows. The rules view groups executable descriptions. All three inspect the same state. A large internal graph includes rule syntax and administrative records as well as the application's data.

3.2 The transaction boundary

Loading a program first parses its source, resolves names and constructs a graph. Validation checks field types, rule ownership, quotation, rank chains and executable revisions before the graph becomes an execution state. Diagnostics retain source context where available. Printing preserves graph structure, rather than the original comments or formatting.

For each step, the engine decodes active rules and searches for injective bindings. It completes the required outgoing context, checks deletion safety, constructs a private candidate state, and validates the result. Rule entry, exit and alteration are then checked against the revision discipline and rank guard. Only admitted occurrences are available to the scheduler.

A successful commit publishes the new graph and its event together. A rejected candidate leaves the original state, including its allocator, unchanged. Prepared work carries a state stamp and becomes stale after a commit or external replacement. This prevents a match prepared against one state from being applied to another.

The general matcher completes its bounded search before selecting an occurrence. Exhaustion is reported separately from an empty enabled set: a search that runs out of resources cannot establish that the program has finished. Indexed and specialised execution paths accelerate recognised program families while retaining their required admission checks. The reference implementation provides a comparison point for these optimisations.

3.3 Scheduling and stopping

The scheduler can select deterministically or use a recorded random seed. Reproducibility depends on the program, implementation, settings and scheduler state. A seeded run does not establish fairness, nor does it imply that renaming atom identities preserves the same selection order.

Execution distinguishes a normal form, a step bound, resource exhaustion and detected recurrence. Recurrence compares observations up to graph isomorphism within the configured checking limits. A repeated observation may omit internal changes, and the scheduler or allocator may have progressed. Its detection is useful evidence about the recorded run, but it does not alone establish a periodic infinite computation.

4. History, explanation and verification

A recorded execution retains its initial graph, settings and committed events. Each event identifies the source rule revision, binding, consumed atoms, preserved context, produced atoms and read footprint. Historical states can therefore be reconstructed and inspected without executing a new rewrite.

Imported traces are untrusted input. The verifier checks their structure and replays their claimed transitions through native admission. Checkpoints retain the execution information needed for supported continuation, including scheduler state. Editing a graph externally establishes a new baseline; it cannot silently become another event in an existing verified history.

The event graph records positive resource dependencies: an earlier event may produce an atom that a later event reads or consumes, or may read an atom that a later event consumes. This supplies a partial order within the recorded execution. Absence conditions and changes in rule visibility can also affect applicability, so the display cannot establish independence merely from a missing edge.

Selected provenance follows the recorded origins of a chosen observation. Bounded counterfactual analyses examine alternatives within an explicitly explored domain. These facilities answer different questions: the former explains the selected history, while the latter tests whether an outcome survives particular interventions. Reports preserve that distinction instead of presenting every dependency as a necessary cause.

5. Browser workbench

The browser sends requests to a worker that owns the WebAssembly execution state. Long-running engine operations therefore do not directly occupy the interface thread. Requests carry generation and request identifiers, allowing the frontend to discard results from a superseded load or execution. Atom

identities cross the JavaScript boundary as decimal strings to avoid loss of integer precision.

The workbench combines a source editor, example catalogue, graph views, atom inspection, rule inspection and execution history. Examples carry their intended settings, which load by default unless the user has overridden them. The lambda editor prepares a graph program through the same compilation route used for its examples.

A timeline selects recorded states. Moving backwards changes the inspected state, and playback advances through existing history. New execution continues from the current endpoint rather than branching invisibly from a historical view. Run controls, rule breakpoints and event inspection support closer examination of individual transitions.

Graph navigation includes zooming, panning, selection and filtering. Rendering and query bounds protect responsiveness on larger states. The interface reports when it shows a subset; filtering does not remove atoms from the execution. This distinction is particularly important when the internal view contains many more atoms than the practical data view.

The guided walkthrough uses one program to connect source, graph structure, rule application and result. Its execution is isolated from the user's current workspace. The Guide provides broader usage instructions, while ordinary screens concentrate on controls and results. Light and dark themes share the same semantic colour conventions.

6. Projects, storage and hosted execution

6.1 Project model

The project API stores immutable source versions, dataset versions and job records. Saving an edited program creates a new version; submitting a job pins that version and its execution settings. Parent links support forks and comparisons without rewriting earlier evidence. These authoring operations are distinct from a rule modifying another rule during execution.

Supported structured datasets compile into ordinary graph programs. Their stored input remains provenance; execution still operates on one carrier containing data and rules. Result reports and paginated event queries help inspect a run, but pagination does not imply that the native engine streams an arbitrarily large graph.

The same API model is available through a local service. Local operation uses a configured bearer token and bounded file storage. Firebase emulators exercise account, database and object-storage integration without requiring the hosted deployment.

6.2 Hosted boundary

Firebase Hosting serves the frontend. Firebase Authentication supplies email accounts, with verification required for API access. A Cloud Run service validates identity and project roles. Firestore holds metadata; Cloud Storage holds source, dataset and trace artefacts. Direct client access to these stores is denied, so application permissions are enforced through the API.

Cloud Tasks dispatches bounded native work. An internal task request uses a separate service identity from an interactive user request. Each batch invokes the Rust executable, verifies its result and publishes a checkpoint or terminal record. The design avoids relying on background computation after an HTTP response.

Idempotency keys identify repeated submissions of the same request. Attempt numbers, storage generations and a transactional owner lease fence stale publication. These controls prevent a delayed worker from replacing the result of a newer attempt. Reconciliation rediscovers durable queued work after an interrupted dispatch.

6.3 Recovery and resource limits

Job lifecycle and graph outcome are recorded separately. A completed job may have reached a step limit rather than a normal form. Pause and resume use verified checkpoints and the original execution identity; an unverified suffix is discarded after interruption. Cancellation is terminal. An eligible interrupted job may resume within its remaining allowance.

Native execution, replay, requests and stored artefacts have explicit bounds. The hosted service serialises useful work through its lease and queue; it is designed for low-volume research and assessment use. Large local experiments have different allowances and cannot be used as evidence that the hosted service accepts the same workload. Application quotas also cannot guarantee a fixed cloud bill.

Hashes, immutable versions and replay verification make corruption or mismatched inputs detectable. They do not establish power-loss safety on every filesystem or complete security against arbitrary workloads. Deployment permissions, backup restoration and execution correctness remain separate engineering concerns.

7. Applications and research interfaces

The platform includes applications that test whether the graph model can express useful computation. Each has an explicit interpretation boundary.

Facility	Integration and scope
Work-item and calendar processing	Structured inputs compile into graph programs. The large calendar pipeline uses the local service and native execution; hosted dataset jobs retain their smaller quotas.
Adaptive workflows	Native recognition checks a restricted task-and-route structure. Persistent-ticket scheduling supports bounded administration and task progress. The project interface can return a contract result; it is not a separate durable workflow-job system.
Lambda evaluation	A compiler represents finite, scoped terms and a closure machine as graph data and ordinary rules. Readback explains closed and explicit-context open weak-head results.
Numerical task analysis	Rust computes checked numerical bounds and publishes results through graph rewrites. The arithmetic is performed by the host application.
Learned models and actions	Rust fitting produces model descriptions or action rules. Resident rules execute the compiled result. Unsafe or contradictory action evidence can lead to an unknown result or quarantine.
Contextual interpretation and calendar proposals	Local adapters construct and assess candidate descriptions under explicit observations and approval conditions. Their learned interpretations remain relative to the task and supplied evidence.
Version-aware execution	A small guest language captures and checks executable versions during supported replacement. It does not allow replacement of the engine’s validator.

Several analyses, including recovery recognition, meta-activity bounds, counterfactual exploration, selected provenance and representation costs, are available through native commands. Their existence in the Rust engine does not imply that each has a browser panel or hosted endpoint. This separation

keeps the public API smaller than the full experimental interface while retaining one underlying implementation.

8. Assurance and limits

Verification combines Rust unit and property-based tests, independent small-case evaluators, cross-runtime comparisons, browser acceptance tests and platform lifecycle tests. Retained experimental inputs, settings and results allow individual claims to be inspected. Performance measurements belong to their recorded builds and workloads; a later implementation does not inherit those measurements automatically.

Lean developments establish selected abstract properties, including parts of the rewriting kernel, bounded search, workflow arguments and evidence analyses. The correspondence between those abstractions and the complete Rust system is supported by explicit reasoning and tests. The project does not claim a machine-checked proof of the whole platform.

The resulting architecture makes a controlled form of reflection practical: executable descriptions are inspectable data, changes pass through one admission boundary, and histories can be checked independently. Its principal limits are the cost of general matching, retained history and lambda-machine garbage, conservative recognisers, and bounded analysis of alternative behaviours. Universal convergence, a globally smallest representation and autonomous acquisition of intrinsic meaning remain research questions beyond the established results.

Software

Reflective Graph source repository. Build instructions, API contracts, executable tests and experimental records are distributed with the software.